Open Source 101 with Angular

How I started contributing to enterprise open source, and how you can do it too.



github.com/erkamyaman



Hi, I'm erKam

`{{ erKam | kæm }}` Döner kebab in developer form 🌯

Working at **Kolay Randevu / SalonAppy**

Contributing to **Angular**, docs and docs-infra

Building **Pangular Inspector**, with Santosh Yadav

Contributing to **Angular Native**

Today

5

Your first contribution

1

How it all started

2

Meaningful things get merged

3

Where I find things to fix

4

How sending a PR to enterprise open source works

My first PR

A Turkish translation of the Angular docs



Small, safe, and useful to Turkish developers

Proof that the process works

Next

How the first review felt, and the moment I thought “I can keep doing this”

The first review

Before

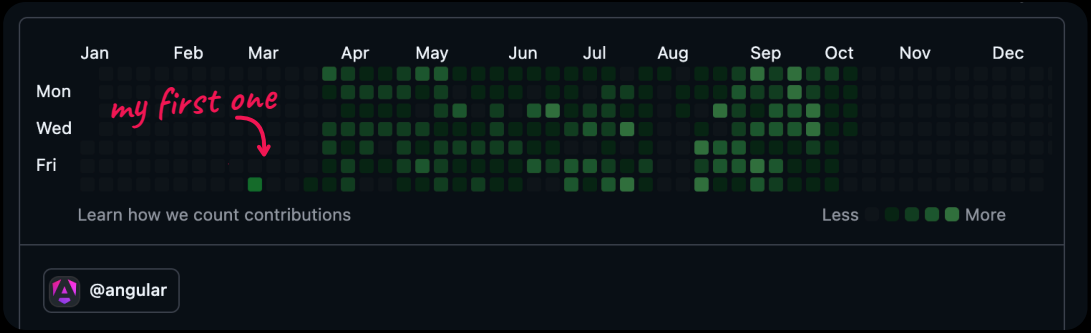
What I felt before clicking “Create pull request”

The wait

How long it took and what the review looked like

After

The moment I thought “I can keep doing this”



What happened next

229 merged PRs

in `angular/angular`

What happened next

My best work is in docs-infra

Plus docs and small fixes across the framework

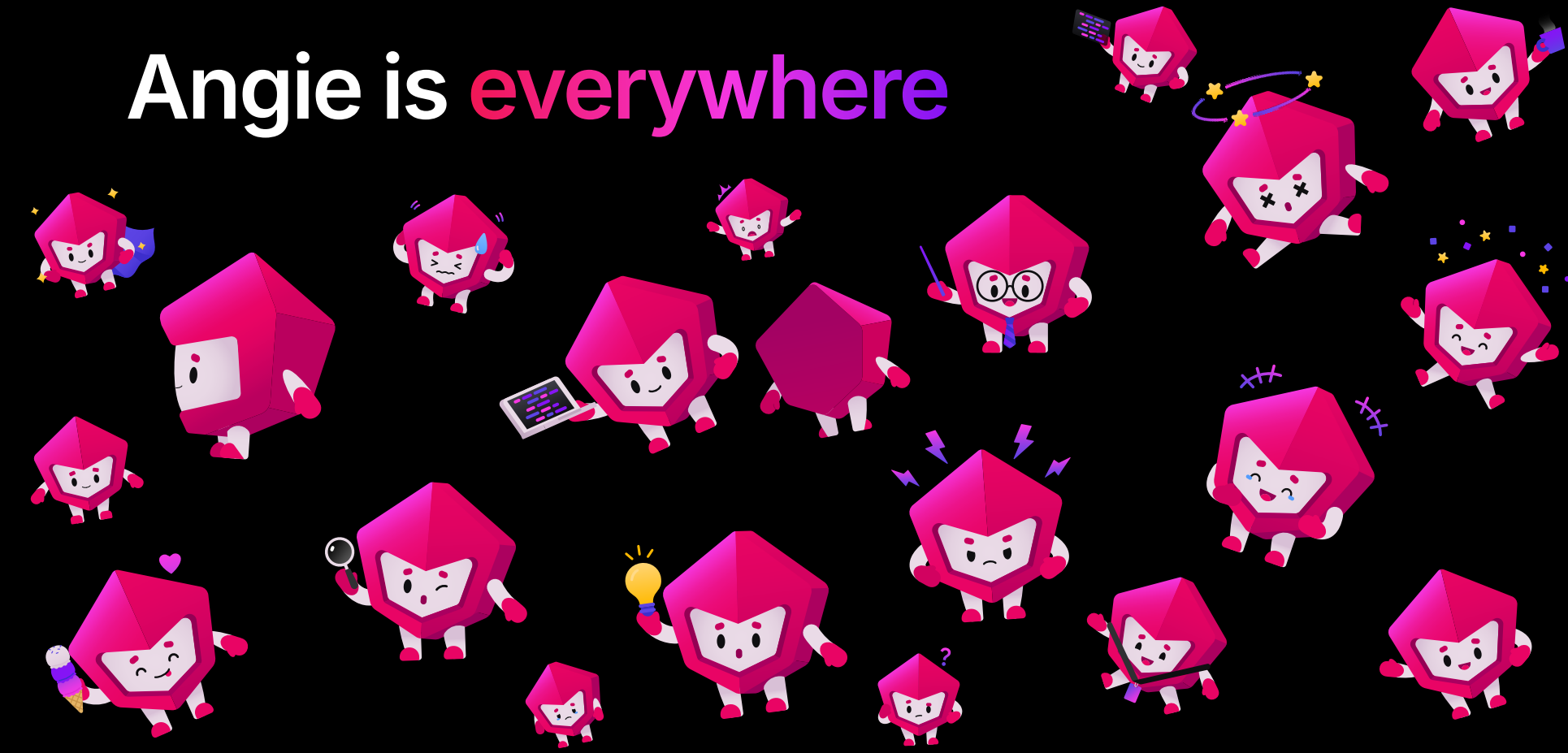
What happened next

The Angular team reached out



Matthieu Riegler challenged me to bring Angie, the Angular mascot, into angular.dev

Angie is everywhere



Angie, the Angular mascot

15+
small PRs

`angular.dev`

Not found page, error states, cookie popup, minigame, tutorials

`Illustrations`

Roadmap, routing, globe, components, guide headers

`DevTools`

Empty states



If you see Angie in a random place on angular.dev, that was probably me

What happened next

Building with Santosh Yadav

Angular GDE and GitHub Star.

Together:  **Pangular Inspector**

What happened next

And more

Angular Native, the great work of Ashley Hunter

What gets merged

229 PRs, 239 commits so far

What I like to send

- Doc fixes 117
- docs-infra bug fixes 62
- Code examples and tutorials 51
- Broken links and anchors 36
- Angie illustrations 17
- docs-infra features 11
- DevTools 8
- Compiler and language tooling 6

What I prefer to skip

- Typos and formatting
- == to ===
- Unused imports
- New docs pages
- Accessibility-only fixes

My workflow

One focused day

Mostly Sundays

I lock in to hunt for issues

With AI

Claude Code

finds bugs, drafts fixes, writes tests

Same bug, other places

Same pattern

if a bug happened once, it is elsewhere too

I treat AI output like a PR from a stranger: I read all of it and reproduce each bug before and after the fix.

Most findings are false positives. The survivors get ranked, and I read every PR locally before I send it.

Angular ships skills for your agent

`adev-writing-guide`

How docs must be written

Google writing rules and Angular markdown

`pr_review`

How a PR gets reviewed

Read all of it, check the commit message

`AGENTS.md`

What an agent may do here

pnpm, Bazel tests, zoneless test style

`angular-developer`

For app developers

best practices for signals, forms, HTTP

☆ The same rules a human follows, written for the agent

Where the bugs come from

Hunting day

Open issues, then the code

I read issues, then the code behind them: angular.dev, packages and the router

Building apps

Hit a bug, open a PR

With Angular Native I also check open Expo and React Native issues, and test on real devices

Poking around

Browse angular.dev by hand

UI and UX bugs I can see in the browser myself

While working

The docs let me down

a broken link, a wrong example or a missing piece

Sending a PR to enterprise open source

Before

Sign the CLA

once, with your commit email

Checks

CI must pass

and the commit message lint

Review

Owners approve

who own that part of the code

Release

Type sets the train

a fix can ship in a patch

Time

Reviews take days

be patient with the team

So

Follow every rule

a small, proven PR passes

Prove it before you open it

Step 1

Reproduce

the claim with a command

Step 2

Check the page

the rendered page, not just the markup

Step 3

Write the PR

with the proof inside

> ☆ A PR with proof gets reviewed. A PR with an opinion gets questions. Reviewers have little time, and evidence saves it.

Preparing the PR

Branch

From upstream main

one fix per branch

Change

Keep it small

only the lines the fix needs

Test

Run the narrow test

not the whole suite

Review

Read your diff

like a PR from a stranger

Commit

Follow the guidelines

type, scope, short header

Description

What, why, proof

like #68625: the bug, before and after screenshots

Commit messages matter

```
fix(docs-infra): correct the quoted error message
```

Type

decides the release train

Scope

is the npm package, not the folder

> ⚠ `feat` cannot land in a patch

Your first contribution

- Pick a project you already use
- Read the contributing guide
- Find a factual error in the docs and prove it
- Open a small PR
- Be patient. Reviews might take days
- Expect feedback. It is not rejection

Bonus: use my first-time contribution skill

Find

Find errors

in the docs, with the page and the line

Prove

Reproduce first

a command, and the rendered page

Prepare

Prepare the PR

branch, commit and PR text

Version 1

Angular version

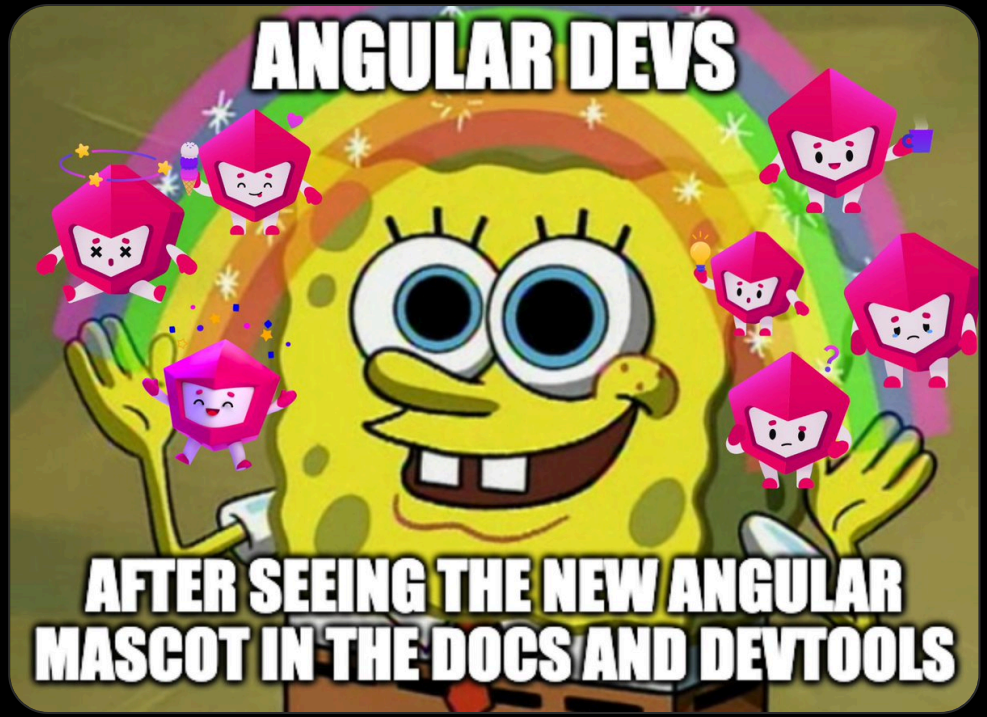
Angular's commit and docs rules

Version 2

Normal version

follows any project's guide

Questions?


ANGULAR DEVS
AFTER SEEING THE NEW ANGULAR MASCOT IN THE DOCS AND DEVTOOLS

Thank you

github.com/erkamyaman

x.com/erkamyaman_ng

linkedin.com/in/erkamyaman



Scan for my links

Made with Slidev

by Anthony Fu and the slidevjs team

Built on Vue

github.com/slidevjs/slidev